

Student Management System — Backend Source Code (Node.js + Express + MySQL)

Single-document export of the full backend codebase for the AWS Cloud & DevOps internship project.

package.json

```
{  
  
  "name": "sms-backend",  
  
  "version": "1.0.0",  
  
  "description": "Student Management System - Backend API (Node.js + Express + MySQL)",  
  
  "main": "src/server.js",  
  
  "scripts": {  
  
    "start": "node src/server.js",  
  
    "dev": "nodemon src/server.js",  
  
    "test": "jest --detectOpenHandles --forceExit"  
  
  },  
  
  "author": "Anil",  
  
  "license": "MIT",  
  
  "dependencies": {  
  
    "bcryptjs": "^2.4.3",  
  
    "cors": "^2.8.5",  
  
    "dotenv": "^16.4.5",
```

```
"express": "^4.19.2",  
  
"express-validator": "^7.2.0",  
  
"helmet": "^7.1.0",  
  
"jsonwebtoken": "^9.0.2",  
  
"morgan": "^1.10.0",  
  
"mysql2": "^3.11.0",  
  
"sequelize": "^6.37.3"  
  
},  
  
"devDependencies": {  
  
  "jest": "^29.7.0",  
  
  "nodemon": "^3.1.4",  
  
  "supertest": "^7.0.0"  
  
}  
  
}
```

.env.example

NODE_ENV=development

PORT=5000

Local docker-compose values shown; in production these point to an

Amazon RDS (MySQL) endpoint and are injected via Kubernetes Secrets.

DB_HOST=mysql

DB_PORT=3306

DB_NAME=sms_db

DB_USER=sms_user

DB_PASSWORD=sms_password

JWT_SECRET=change_this_to_a_long_random_string

src/config/db.js

```
const { Sequelize } = require('sequelize');
```

```
require('dotenv').config();
```

```
// Database connection is configured entirely through environment variables so
```

```
// the same image can target local MySQL (docker-compose) or Amazon RDS in
```

```
// production without any code changes (12-factor app principle).
```

```
const sequelize = new Sequelize(
```

```
  process.env.DB_NAME || 'sms_db',
```

```
  process.env.DB_USER || 'sms_user',
```

```
  process.env.DB_PASSWORD || 'sms_password',
```

```
{
```

```
  host: process.env.DB_HOST || 'localhost',
```

```
  port: process.env.DB_PORT || 3306,
```

```
  dialect: 'mysql',
```

```
  logging: process.env.NODE_ENV === 'development' ? console.log : false,
```

```
  pool: {
```

```
    max: 10,
```

```
    min: 0,  
  
    acquire: 30000,  
  
    idle: 10000  
  
  }  
  
}  
  
);  
  
module.exports = sequelize;
```

src/models/index.js

```
const sequelize = require('../config/db');  
  
const User = require('./User');  
  
const Course = require('./Course');  
  
const Student = require('./Student');  
  
const Enrollment = require('./Enrollment');  
  
module.exports = {  
  
  sequelize,  
  
  User,  
  
  Course,  
  
  Student,  
  
  Enrollment  
  
};
```

src/models/User.js

```
const { DataTypes } = require('sequelize');
```

```
const bcrypt = require('bcryptjs');
```

```
const sequelize = require('../config/db');
```

```
const User = sequelize.define('User', {
```

```
  id: {
```

```
    type: DataTypes.INTEGER,
```

```
    primaryKey: true,
```

```
    autoIncrement: true
```

```
  },
```

```
  name: {
```

```
    type: DataTypes.STRING,
```

```
    allowNull: false
```

```
  },
```

```
  email: {
```

```
    type: DataTypes.STRING,
```

```
    allowNull: false,
```

```
    unique: true,
```

```
    validate: { isEmail: true }
```

```
  },
```

```
  password: {
```

```

    type: DataTypes.STRING,

    allowNull: false

  },

  role: {

    type: DataTypes.ENUM('admin', 'teacher', 'student'),

    defaultValue: 'student'

  }

}, {

  tableName: 'users',

  timestamps: true,

  hooks: {

    beforeCreate: async (user) => {

      user.password = await bcrypt.hash(user.password, 10);

    },

    beforeUpdate: async (user) => {

      if (user.changed('password')) {

        user.password = await bcrypt.hash(user.password, 10);

      }

    }

  }

});

User.prototype.comparePassword = function (candidatePassword) {

```

```
    return bcrypt.compare(candidatePassword, this.password);  
  };  
  
  module.exports = User;
```

src/models/Course.js

```
const { DataTypes } = require('sequelize');  
  
const sequelize = require('../config/db');  
  
const Course = sequelize.define('Course', {  
  
  id: {  
  
    type: DataTypes.INTEGER,  
  
    primaryKey: true,  
  
    autoIncrement: true  
  
  },  
  
  code: {  
  
    type: DataTypes.STRING,  
  
    allowNull: false,  
  
    unique: true  
  
  },  
  
  name: {  
  
    type: DataTypes.STRING,  
  
    allowNull: false  
  
  },  
  
});
```

```

    durationMonths: {
      type: DataTypes.INTEGER,
      allowNull: false,
      defaultValue: 12
    },
    description: {
      type: DataTypes.TEXT,
      allowNull: true
    }
  }, {
    tableName: 'courses',
    timestamps: true
  });

module.exports = Course;

```

src/models/Student.js

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/db');

const Course = require('./Course');

const User = require('./User');

const Student = sequelize.define('Student', {
  id: {

```



```
    type: DataTypes.INTEGER,  
  
    primaryKey: true,  
  
    autoIncrement: true  
  },
```

```
  rollNumber: {  
  
    type: DataTypes.STRING,  
  
    allowNull: false,  
  
    unique: true  
  },
```

```
  name: {  
  
    type: DataTypes.STRING,  
  
    allowNull: false  
  },
```

```
  email: {  
  
    type: DataTypes.STRING,  
  
    allowNull: false,  
  
    unique: true,  
  
    validate: { isEmail: true }  
  },
```

```
  phone: {  
  
    type: DataTypes.STRING,
```

```
    allowNull: true

  },

  dateOfBirth: {

    type: DataTypes.DATEONLY,

    allowNull: true

  },

  address: {

    type: DataTypes.STRING,

    allowNull: true

  },

  status: {

    type: DataTypes.ENUM('active', 'inactive', 'graduated'),

    defaultValue: 'active'

  },

  courseId: {

    type: DataTypes.INTEGER,

    allowNull: true,

    references: { model: Course, key: 'id' }

  },

  userId: {

    type: DataTypes.INTEGER,

    allowNull: true,
```

```

    references: { model: User, key: 'id' }

  }, {

    tableName: 'students',

    timestamps: true

  });

Student.belongsTo(Course, { foreignKey: 'courseId', as: 'course' });

Course.hasMany(Student, { foreignKey: 'courseId', as: 'students' });

Student.belongsTo(User, { foreignKey: 'userId', as: 'user' });

module.exports = Student;

```

src/models/Enrollment.js

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/db');

const Student = require('./Student');

const Course = require('./Course');

const Enrollment = sequelize.define('Enrollment', {

  id: {

    type: DataTypes.INTEGER,

    primaryKey: true,

    autoIncrement: true

  },

```

```
studentId: {  
  
  type: DataTypes.INTEGER,  
  
  allowNull: false,  
  
  references: { model: Student, key: 'id' }  
  
},  
  
courseId: {  
  
  type: DataTypes.INTEGER,  
  
  allowNull: false,  
  
  references: { model: Course, key: 'id' }  
  
},  
  
enrollmentDate: {  
  
  type: DataTypes.DATEONLY,  
  
  defaultValue: DataTypes.NOW  
  
},  
  
grade: {  
  
  type: DataTypes.STRING,  
  
  allowNull: true  
  
},  
  
status: {  
  
  type: DataTypes.ENUM('enrolled', 'completed', 'dropped'),  
  
  defaultValue: 'enrolled'
```

```

    }
  }, {
    tableName: 'enrollments',

    timestamps: true

  });

// onDelete CASCADE belongs on the belongsTo() side in Sequelize, since that
// association owns the foreign key column - this was found and corrected

// during local testing (a plain hasMany-side onDelete is silently ignored).

Enrollment.belongsTo(Student, { foreignKey: 'studentId', as: 'student', onDelete: 'CASCADE' });

Enrollment.belongsTo(Course, { foreignKey: 'courseId', as: 'course', onDelete: 'CASCADE' });

Student.hasMany(Enrollment, { foreignKey: 'studentId', as: 'enrollments' });

Course.hasMany(Enrollment, { foreignKey: 'courseId', as: 'enrollments' });

module.exports = Enrollment;

```

src/middleware/auth.js

```

const jwt = require('jsonwebtoken');

// Verifies the Bearer token on protected routes and attaches the decoded
// payload (id, role) to req.user.

function authenticate(req, res, next) {

  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {

    return res.status(401).json({ message: 'Authorization token missing' });
  }
}

```

```

}

const token = authHeader.split(' ')[1];

try {

  const decoded = jwt.verify(token, process.env.JWT_SECRET || 'dev_secret_change_me');

  req.user = decoded;

  next();

} catch (err) {

  return res.status(401).json({ message: 'Invalid or expired token' });

}

}

// Restricts a route to one or more roles, e.g. authorize('admin')

function authorize(...roles) {

  return (req, res, next) => {

    if (!req.user || !roles.includes(req.user.role)) {

      return res.status(403).json({ message: 'Insufficient permissions' });

    }

    next();

  };

}

module.exports = { authenticate, authorize };

```

src/controllers/authController.js

```
const jwt = require('jsonwebtoken');

const { validationResult } = require('express-validator');

const { User } = require('../models');

function signToken(user) {

  return jwt.sign(

    { id: user.id, role: user.role, name: user.name },

    process.env.JWT_SECRET || 'dev_secret_change_me',

    { expiresIn: '8h' }

  );

}

exports.register = async (req, res) => {

  const errors = validationResult(req);

  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

  try {

    const { name, email, password, role } = req.body;

    const existing = await User.findOne({ where: { email } });

    if (existing) return res.status(409).json({ message: 'Email already registered' });

    const user = await User.create({ name, email, password, role: role || 'student' });

    const token = signToken(user);

    res.status(201).json({
```

```

    message: 'User registered successfully',

    token,

    user: { id: user.id, name: user.name, email: user.email, role: user.role }

  });

} catch (err) {

  res.status(500).json({ message: 'Registration failed', error: err.message });

}

};

exports.login = async (req, res) => {

  const errors = validationResult(req);

  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

  try {

    const { email, password } = req.body;

    const user = await User.findOne({ where: { email } });

    if (!user) return res.status(401).json({ message: 'Invalid credentials' });

    const match = await user.comparePassword(password);

    if (!match) return res.status(401).json({ message: 'Invalid credentials' });

    const token = signToken(user);

    res.json({

      message: 'Login successful',

      token,

      user: { id: user.id, name: user.name, email: user.email, role: user.role }
    });
  } catch (err) {
    res.status(500).json({ message: 'Internal server error', error: err.message });
  }
};

```



```

    });

    } catch (err) {

        res.status(500).json({ message: 'Login failed', error: err.message });

    }

};

exports.me = async (req, res) => {

    const user = await User.findByPk(req.user.id, { attributes: ['id', 'name', 'email', 'role'] });

    res.json(user);

};

```

src/controllers/studentController.js

```

const { validationResult } = require('express-validator');

const { Student, Course } = require('../models');

exports.getAllStudents = async (req, res) => {

    try {

        const { page = 1, limit = 10, search = '' } = req.query;

        const offset = (page - 1) * limit;

        const { Op } = require('sequelize');

        const where = search

        ? {

            [Op.or]: [

                { name: { [Op.like]: `%${search}%` } },

```

```

      { rollNumber: { [Op.like]: `%${search}%` } },

      { email: { [Op.like]: `%${search}%` } }

    ]

  }

  : {});

const { rows, count } = await Student.findAndCountAll({

  where,

  include: [{ model: Course, as: 'course', attributes: ['id', 'name', 'code'] }],

  limit: parseInt(limit),

  offset: parseInt(offset),

  order: [['createdAt', 'DESC']]

});

res.json({ total: count, page: parseInt(page), limit: parseInt(limit), students: rows });

} catch (err) {

  res.status(500).json({ message: 'Failed to fetch students', error: err.message });

}

};

exports.getStudentById = async (req, res) => {

  try {

    const student = await Student.findByPk(req.params.id, {

      include: [{ model: Course, as: 'course' }]

    });

    if (!student) {

      res.status(404).json({ message: 'Student not found' });

      return;

    }

    res.json(student);

  } catch (err) {

    res.status(500).json({ message: 'Failed to fetch student', error: err.message });

  }

};

```

```
});

if (!student) return res.status(404).json({ message: 'Student not found' });

res.json(student);

} catch (err) {

  res.status(500).json({ message: 'Failed to fetch student', error: err.message });

}

};

exports.createStudent = async (req, res) => {

  const errors = validationResult(req);

  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

  try {

    const student = await Student.create(req.body);

    res.status(201).json({ message: 'Student created successfully', student });

  } catch (err) {

    res.status(500).json({ message: 'Failed to create student', error: err.message });

  }

};

exports.updateStudent = async (req, res) => {

  try {

    const student = await Student.findByIdPk(req.params.id);

    if (!student) return res.status(404).json({ message: 'Student not found' });

    await student.update(req.body);
```

```

    res.json({ message: 'Student updated successfully', student });

  } catch (err) {

    res.status(500).json({ message: 'Failed to update student', error: err.message });

  }

};

exports.deleteStudent = async (req, res) => {

  try {

    const student = await Student.findByIdPk(req.params.id);

    if (!student) return res.status(404).json({ message: 'Student not found' });

    await student.destroy();

    res.json({ message: 'Student deleted successfully' });

  } catch (err) {

    res.status(500).json({ message: 'Failed to delete student', error: err.message });

  }

};

```

src/controllers/courseController.js

```

const { validationResult } = require('express-validator');

const { Course, Student } = require('../models');

exports.getAllCourses = async (req, res) => {

  try {

    const courses = await Course.findAll({ order: [['createdAt', 'DESC']] });

```

```

    res.json(courses);

  } catch (err) {

    res.status(500).json({ message: 'Failed to fetch courses', error: err.message });

  }

};

exports.getCourseById = async (req, res) => {

  try {

    const course = await Course.findByPk(req.params.id, {

      include: [{ model: Student, as: 'students', attributes: ['id', 'name', 'rollNumber'] }]

    });

    if (!course) return res.status(404).json({ message: 'Course not found' });

    res.json(course);

  } catch (err) {

    res.status(500).json({ message: 'Failed to fetch course', error: err.message });

  }

};

exports.createCourse = async (req, res) => {

  const errors = validationResult(req);

  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

  try {

    const course = await Course.create(req.body);

```

```
    res.status(201).json({ message: 'Course created successfully', course });

  } catch (err) {

    res.status(500).json({ message: 'Failed to create course', error: err.message });

  }

};

exports.updateCourse = async (req, res) => {

  try {

    const course = await Course.findByPk(req.params.id);

    if (!course) return res.status(404).json({ message: 'Course not found' });

    await course.update(req.body);

    res.json({ message: 'Course updated successfully', course });

  } catch (err) {

    res.status(500).json({ message: 'Failed to update course', error: err.message });

  }

};

exports.deleteCourse = async (req, res) => {

  try {

    const course = await Course.findByPk(req.params.id);

    if (!course) return res.status(404).json({ message: 'Course not found' });

    await course.destroy();

    res.json({ message: 'Course deleted successfully' });

  } catch (err) {
```

```
    res.status(500).json({ message: 'Failed to delete course', error: err.message });  
  
  }  
  
};
```

src/controllers/enrollmentController.js

```
const { Enrollment, Student, Course } = require('../models');  
  
exports.getAllEnrollments = async (req, res) => {  
  
  try {  
  
    const enrollments = await Enrollment.findAll({  
  
      include: [  
  
        { model: Student, as: 'student', attributes: ['id', 'name', 'rollNumber'] },  
  
        { model: Course, as: 'course', attributes: ['id', 'name', 'code'] }  
  
      ],  
  
      order: [['createdAt', 'DESC']]  
  
    });  
  
    res.json(enrollments);  
  
  } catch (err) {  
  
    res.status(500).json({ message: 'Failed to fetch enrollments', error: err.message });  
  
  }  
  
};  
  
exports.createEnrollment = async (req, res) => {  
  
  try {
```

```
const { studentId, courseId } = req.body;

const student = await Student.findById(studentId);

if (!student) return res.status(404).json({ message: 'Student not found' });

const course = await Course.findById(courseId);

if (!course) return res.status(404).json({ message: 'Course not found' });

const enrollment = await Enrollment.create(req.body);

res.status(201).json({ message: 'Enrollment created successfully', enrollment });

} catch (err) {

  res.status(500).json({ message: 'Failed to create enrollment', error: err.message });

}

};

exports.updateEnrollment = async (req, res) => {

  try {

    const enrollment = await Enrollment.findById(req.params.id);

    if (!enrollment) return res.status(404).json({ message: 'Enrollment not found' });

    await enrollment.update(req.body);

    res.json({ message: 'Enrollment updated successfully', enrollment });

  } catch (err) {

    res.status(500).json({ message: 'Failed to update enrollment', error: err.message });

  }

};
```



```
exports.deleteEnrollment = async (req, res) => {  
  
  try {  
  
    const enrollment = await Enrollment.findByPk(req.params.id);  
  
    if (!enrollment) return res.status(404).json({ message: 'Enrollment not found' });  
  
    await enrollment.destroy();  
  
    res.json({ message: 'Enrollment deleted successfully' });  
  
  } catch (err) {  
  
    res.status(500).json({ message: 'Failed to delete enrollment', error: err.message });  
  
  }  
  
};
```

src/routes/healthRoutes.js

```
const express = require('express');  
  
const router = express.Router();  
  
const sequelize = require('../config/db');  
  
// Used by the ALB target group health check and the Kubernetes  
  
// liveness/readiness probes.  
  
router.get('/', async (req, res) => {  
  
  try {  
  
    await sequelize.authenticate();  
  
    res.status(200).json({ status: 'ok', database: 'connected', timestamp: new Date().toISOString() });  
  
  } catch (err) {
```

```
    res.status(503).json({ status: 'error', database: 'disconnected', error: err.message });

  }

});

module.exports = router;
```

src/routes/authRoutes.js

```
const express = require('express');

const { body } = require('express-validator');

const router = express.Router();

const authController = require('../../controllers/authController');

const { authenticate } = require('../../middleware/auth');

router.post(

  '/register',

  [

    body('name').notEmpty().withMessage('Name is required'),

    body('email').isEmail().withMessage('Valid email is required'),

    body('password').isLength({ min: 6 }).withMessage('Password must be at least 6 characters')

  ],

  authController.register

);

router.post(

  '/login',
```

```
[
  body('email').isEmail().withMessage('Valid email is required'),
  body('password').notEmpty().withMessage('Password is required')
],
authController.login
);

router.get('/me', authenticate, authController.me);

module.exports = router;
```

src/routes/studentRoutes.js

```
const express = require('express');

const { body } = require('express-validator');

const router = express.Router();

const studentController = require('../controllers/studentController');

const { authenticate, authorize } = require('../middleware/auth');

router.use(authenticate);

router.get('/', studentController.getAllStudents);

router.get('/:id', studentController.getStudentById);

router.post(
  '/',
  authorize('admin', 'teacher'),
  [
```

```
    body('name').notEmpty().withMessage('Name is required'),

    body('rollNumber').notEmpty().withMessage('Roll number is required'),

    body('email').isEmail().withMessage('Valid email is required')

],

studentController.createStudent

);

router.put('/:id', authorize('admin', 'teacher'), studentController.updateStudent);

router.delete('/:id', authorize('admin'), studentController.deleteStudent);

module.exports = router;
```

src/routes/courseRoutes.js

```
const express = require('express');

const { body } = require('express-validator');

const router = express.Router();

const courseController = require('../../controllers/courseController');

const { authenticate, authorize } = require('../../middleware/auth');

router.use(authenticate);

router.get('/', courseController.getAllCourses);

router.get('/:id', courseController.getCourseById);

router.post(

  '/',

  authorize('admin', 'teacher'),
```

```
[
  body('name').notEmpty().withMessage('Course name is required'),
  body('code').notEmpty().withMessage('Course code is required')
],
courseController.createCourse
);

router.put('/:id', authorize('admin', 'teacher'), courseController.updateCourse);

router.delete('/:id', authorize('admin'), courseController.deleteCourse);

module.exports = router;
```

src/routes/enrollmentRoutes.js

```
const express = require('express');

const router = express.Router();

const enrollmentController = require('../../controllers/enrollmentController');

const { authenticate, authorize } = require('../../middleware/auth');

router.use(authenticate);

router.get('/', enrollmentController.getAllEnrollments);

router.post('/', authorize('admin', 'teacher'), enrollmentController.createEnrollment);

router.put('/:id', authorize('admin', 'teacher'), enrollmentController.updateEnrollment);

router.delete('/:id', authorize('admin'), enrollmentController.deleteEnrollment);

module.exports = router;
```

src/app.js

```
const express = require('express');

const cors = require('cors');

const helmet = require('helmet');

const morgan = require('morgan');

const authRoutes = require('./routes/authRoutes');

const studentRoutes = require('./routes/studentRoutes');

const courseRoutes = require('./routes/courseRoutes');

const enrollmentRoutes = require('./routes/enrollmentRoutes');

const healthRoutes = require('./routes/healthRoutes');

const app = express();

app.use(helmet());

app.use(cors());

app.use(express.json());

app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev'));

app.use('/health', healthRoutes);

app.use('/api/auth', authRoutes);

app.use('/api/students', studentRoutes);

app.use('/api/courses', courseRoutes);

app.use('/api/enrollments', enrollmentRoutes);

app.get('/', (req, res) => {
```

```
    res.json({ message: 'Student Management System API', version: '1.0.0' });
  });

// 404 handler

app.use((req, res) => {

  res.status(404).json({ message: 'Route not found' });

});

// Centralized error handler

app.use((err, req, res, next) => {

  console.error(err.stack);

  res.status(err.status || 500).json({ message: err.message || 'Internal server error' });

});

module.exports = app;
```

src/server.js

```
require('dotenv').config();

const app = require('./app');

const { sequelize } = require('./models');

const PORT = process.env.PORT || 5000;

async function start() {

  try {

    await sequelize.authenticate();

    console.log('Database connection established successfully.');
```

```
// In production, real migrations (sequelize-cli) should be used instead

// of sync(); alter:true is convenient for this internship project scope.

await sequelize.sync({ alter: true });

console.log('Database models synchronized.');
```

```
app.listen(PORT, () => {

  console.log(`SMS backend API running on port ${PORT}`);

});

} catch (err) {

  console.error('Unable to start server:', err.message);

  process.exit(1);

}

}

start();
```